

Parallel computing projet : Meet In The Middle Attack

Mathieu Laurent

Victor Pezenec-Deutsch

January 2026

1 Introduction

This report presents the design, implementation, and performance evaluation of a parallel Meet-in-the-Middle (MitM) attack on the SPECK block cipher. The objective of this work was to design a scalable implementation capable of solving large instances ($n \geq 34$), which are intractable on a single machine due to the $O(2^{n/2})$ memory and time complexity.

The implementation evolved through two main stages in order to address the performance bottlenecks encountered at scale:

1. **Pure MPI Strategy:** a baseline distributed implementation relying exclusively on MPI for both computation and communication.
2. **Hybrid MPI+OpenMP Strategy:** a redesigned approach separating computation and communication to better exploit shared-memory parallelism within nodes.

Using the final hybrid approach, we successfully solved instances up to $n = 39$ using up to 80 MPI processes.

2 Experimental Setup

All experiments were conducted on the Grid5000 cluster (Nancy site). Each node provides 18 cores and 96GB of RAM. Compilation flags, resource reservation, and execution commands are detailed in the accompanying README file.

3 First Strategy: Sharding with MPI

To parallelize the meet-in-the-middle attack, the key space is distributed across P MPI processes using a cyclic scheme. Each process of rank r iterates over all keys x such that:

$$x \equiv r \pmod{P}.$$

This rule is applied consistently during both Phase 1 (dictionary construction) and Phase 2 (search phase), ensuring a balanced distribution of the 2^n candidates.

The dictionary itself is distributed using a second sharding rule based on the value of $f(x)$. For each computed value $z = f(x)$, ownership is determined by:

$$\text{owner}(z) = \text{murmur64}(z) \pmod{P}.$$

If $\text{owner}(z) = r$, the pair (z, x) is inserted locally; otherwise, it is appended to a send buffer associated with the destination process. These buffers are periodically flushed using batched `MPI_Send` operations.

On the receiving side, each process continuously probes for incoming messages, unpacks received pairs, and inserts them into its local hash table. This scheme guarantees that all identical values of $f(x)$ are stored on the same process, ensuring that any collision $f(k_1) = g(k_2)$ can be detected locally.

To avoid synchronous point-to-point queries during Phase 2, requests for remote dictionary lookups are batched and sent using tag 20. Matching results are returned using tag 21. This asynchronous batching strategy ensures correctness while limiting communication overhead.

3.1 Performance Analysis

Table 1 reports execution time, communication time, and overhead for several values of n and P , using a fixed batch size of 2048.

Table 1: Parallel performance of the MPI-only implementation (batch size: 2048)

P	n	Mem/proc	Total (s)	Comm (s)	Overhead (s)
4	25	113.2MB	15.5	5.4	4.8
4	26	226.5MB	32.2	11.5	9.8
4	27	453.0MB	65.8	24.9	18.8
4	28	906.0MB	132.8	51.4	37.0
4	29	1.8GB	268.8	112.2	68.2
4	30	3.6GB	530.7	207.4	146.1
8	25	56.6MB	13.1	5.5	4.9
8	26	113.2MB	28.3	11.7	10.7
8	27	226.5MB	47.6	20.2	16.2
8	28	453.0MB	125.8	51.5	51.3
8	29	906.0MB	240.4	103.6	90.4
8	30	1.8GB	484.0	200.2	191.3
16	25	28.3MB	12.7	5.9	5.3
16	26	56.6MB	30.5	13.3	14.1
16	27	113.2MB	66.3	28.2	31.7
16	28	226.5MB	139.0	57.9	67.9
16	29	453.0MB	283.0	117.0	139.6
16	30	906.0MB	567.1	235.5	279.1

Increasing the number of processes from $P = 4$ to $P = 8$ reduces execution time. However, moving from $P = 8$ to $P = 16$ degrades performance: for $n = 30$, the total execution time increases from 484.0 s to 567.1 s. This slowdown is explained by the dense communication pattern induced by the sharding rule $\text{owner}(z) = \text{murmur64}(z) \bmod P$, which increases the number of peer connections and synchronization events as P grows.

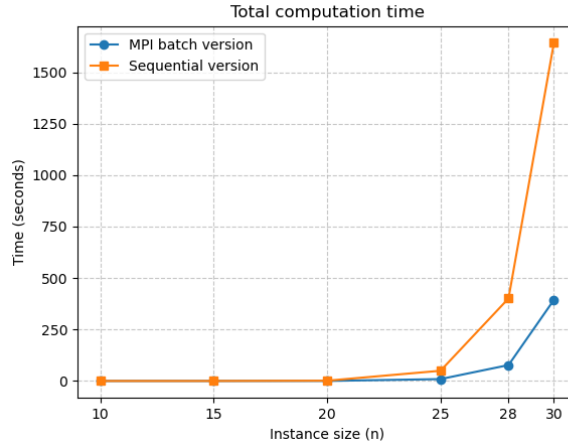


Figure 1: Comparison between the sequential algorithm and the MPI batch version

Figures 2 and 3 confirm this behavior: the efficiency drops to approximately 2% for $P = 16$, indicating a regime of diminishing returns where communication and synchronization dominate computation.

3.1.1 Batch Size Impact

Batching amortizes communication latency by reducing the number of messages. Once a minimal batch size is reached, communication costs are dominated by bandwidth rather than latency. As a

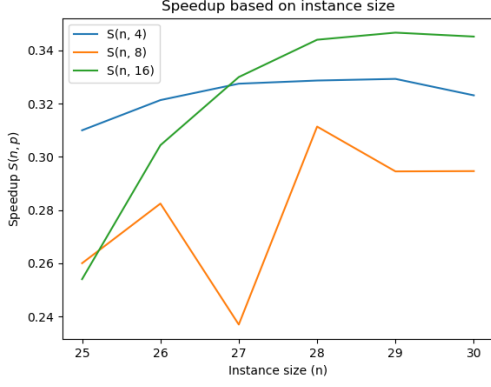


Figure 2: Speedup (MPI Only)

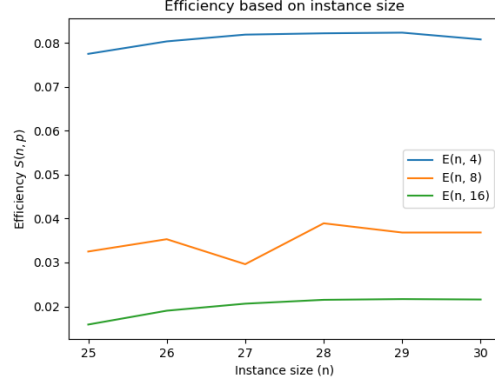


Figure 3: Efficiency (MPI Only)

result, for large instances ($n = 30$), varying the batch size has a limited impact on total execution time (less than 9% variation), indicating that batching is necessary but not the primary scalability bottleneck.

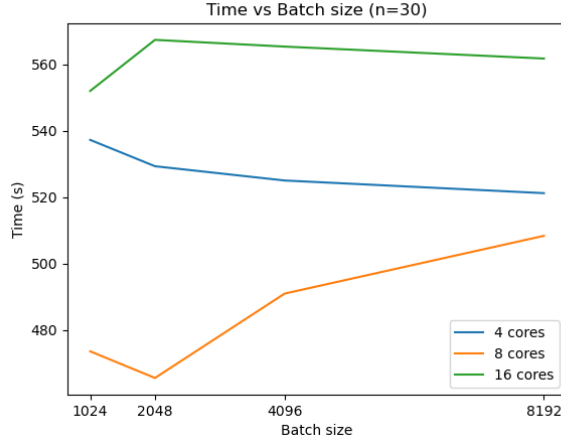


Figure 4: Time vs batch size on instance $n=30$

4 Hybrid Parallelization: MPI and OpenMP

To overcome the limitations of the MPI-only strategy, we redesigned the implementation using a hybrid MPI+OpenMP approach. MPI communications are handled exclusively by the master thread using the `MPI_THREAD_FUNNELED` mode, while OpenMP threads parallelize the cryptographic computations of $f(x)$ and $g(y)$.

This separation decouples computation from communication, reduces contention, and allows each node to fully exploit shared-memory parallelism.

4.1 MPI + OpenMP Performance Analysis

Figure 5 presents the parallel efficiency for 1, 2, and 3 compute nodes. The near overlap of the curves shows that performance is largely independent of the number of nodes, indicating that inter-node communication is no longer the dominant bottleneck.

Figure 6 shows that speedup remains sublinear due to residual synchronization and dictionary management overheads. However, increasing the problem size improves efficiency, as the computational workload better amortizes parallel overhead.

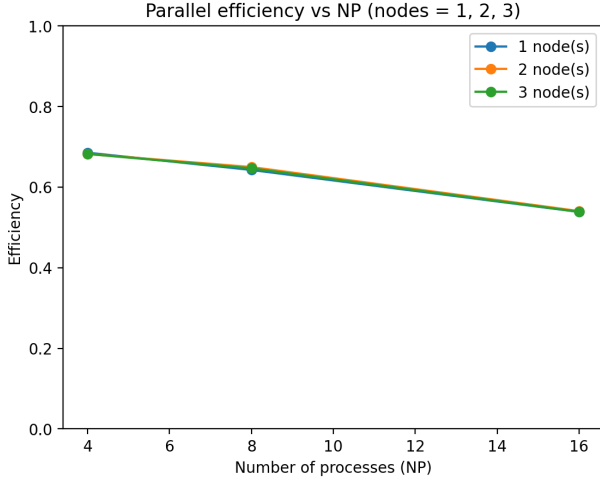


Figure 5: Parallel efficiency vs NP for 1, 2, and 3 nodes.

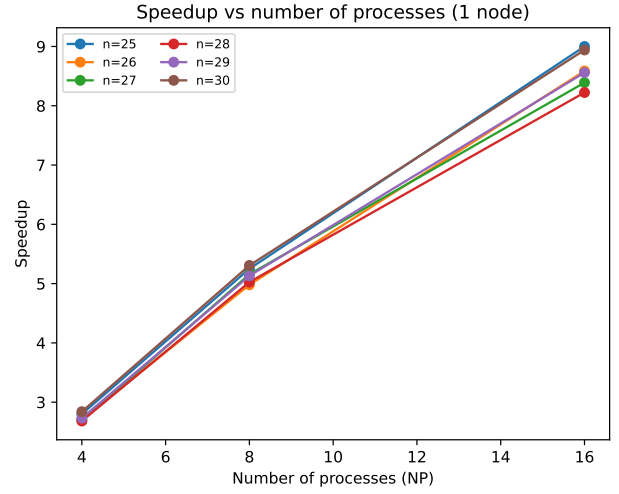


Figure 6: Speedup vs NP on a single node.

Table 2 confirms that increasing the number of nodes with a fixed number of MPI processes does not significantly reduce execution time, further indicating algorithmic rather than hardware limitations.

Table 2: Measured speedup for NP=4 as a function of nodes and n .

n	1 node	2 nodes	3 nodes
25	2.80	2.74	2.80
26	2.71	2.68	2.68
27	2.69	2.66	2.70
28	2.68	2.73	2.66
29	2.73	2.75	2.72
30	2.84	2.82	2.81

5 Large-Scale Results ($n = 34$ to 39)

Table 3 summarizes the largest instances solved using the hybrid implementation. For $n = 34, 35$, and 36, the number of MPI processes was kept constant ($P = 15$), validating the expected $O(2^n)$ time complexity. Execution time approximately doubles when n increases by one.

For $n \geq 37$, memory requirements exceed the capacity of 15 processes, and the number of MPI processes was increased accordingly. For $n = 39$, the search space is 32 times larger than for $n = 34$. Using 80 MPI processes, the instance was solved in 4410 seconds, demonstrating excellent weak scalability.

Table 3: Performance of the Hybrid MPI+OpenMP attack

n	2^n	MPI	Threads	Cores	Time (s)	Mem/proc
34	1.7×10^{10}	15	8	120	684.4	15.5 GB
35	3.4×10^{10}	15	8	120	1415.3	30.9 GB
36	6.8×10^{10}	15	8	120	2907.1	61.8 GB
37	1.3×10^{11}	25	8	200	3460.8	74.2 GB
38	2.7×10^{11}	45	8	360	3869.0	82.5 GB
39	5.5×10^{11}	80	8	640	4410.1	92.8 GB

6 Conclusion and Perspectives

This work demonstrates that a hybrid MPI+OpenMP approach significantly improves scalability compared to a pure MPI implementation for large-scale meet-in-the-middle attacks. The main limiting factor remains memory consumption: for $n = 39$, the distributed dictionary requires approximately 93 GB per MPI process.

To overcome this limitation and reach larger instances (e.g., $n \geq 50$), a time-memory trade-off strategy is required. By partitioning the search space into smaller subsets, memory usage can be reduced at the cost of increased computation time, effectively transforming the problem from memory-bound to compute-bound.

A Verified Solutions

In this section, you will find all the keys obtained during our experiments. We also provide the URL from which our code was executed in order to find these keys.

<https://ppar.tme-crypto.fr/toto/42>

Table 4: Golden Claws found for selected instances (verification only)

n	MPI Rank	Collision Pair (Hexadecimal)	
		$k_1(x)$	$k_2(z)$
26	–	0000000001f27668	00000000090f25c
27	–	0000000002834709	00000000014840bc
28	–	00000000065078f6	00000000044a0d3a
29	–	0000000009251c50	000000000b2a1570
30	–	000000001d95b7ec	000000000a4bb630
34	4	000000017fe4dfd2	00000000e1e41030
35	14	00000001dae10faa	00000000394a7055
36	0	0000000218d1b6c9	0000000c98428058
37	24	000000189d80df86	00000004270dcf81
38	19	00000035e393e6ac	0000002e947d2271
39	73	000000400be27c74	0000004650adbb91